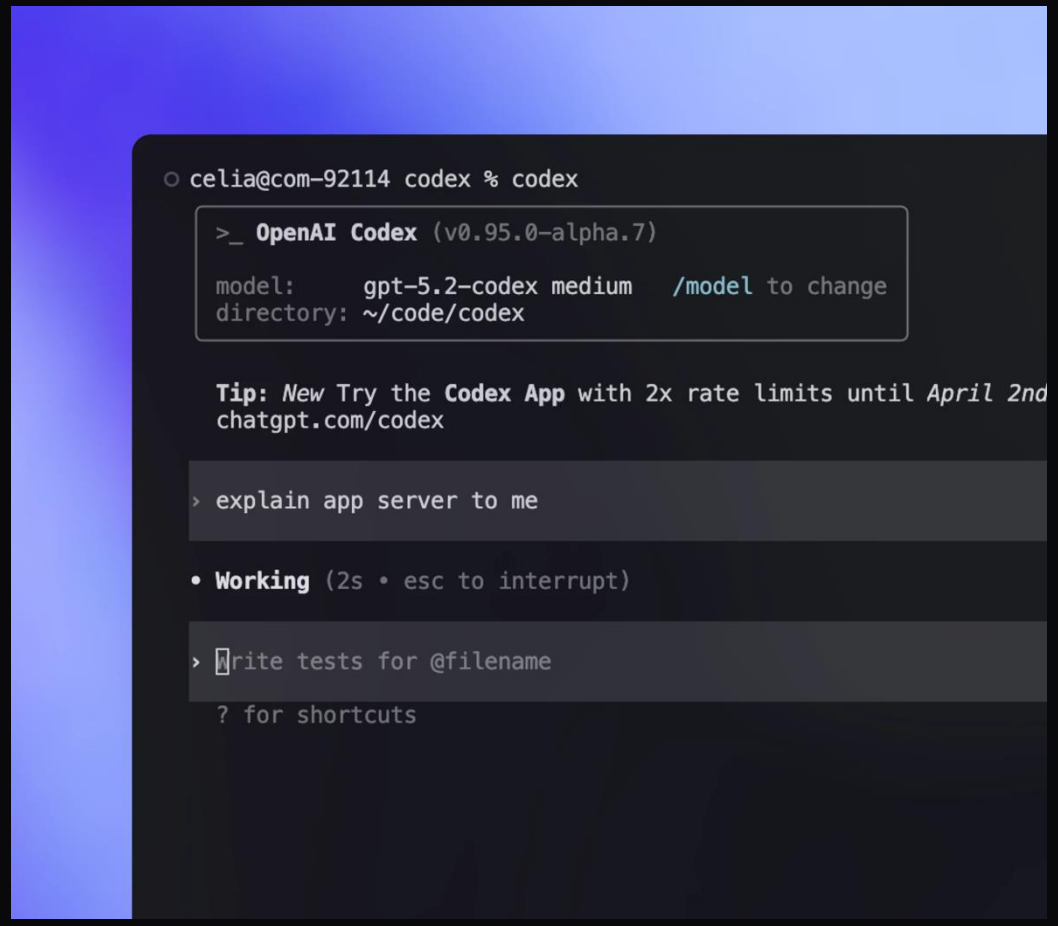
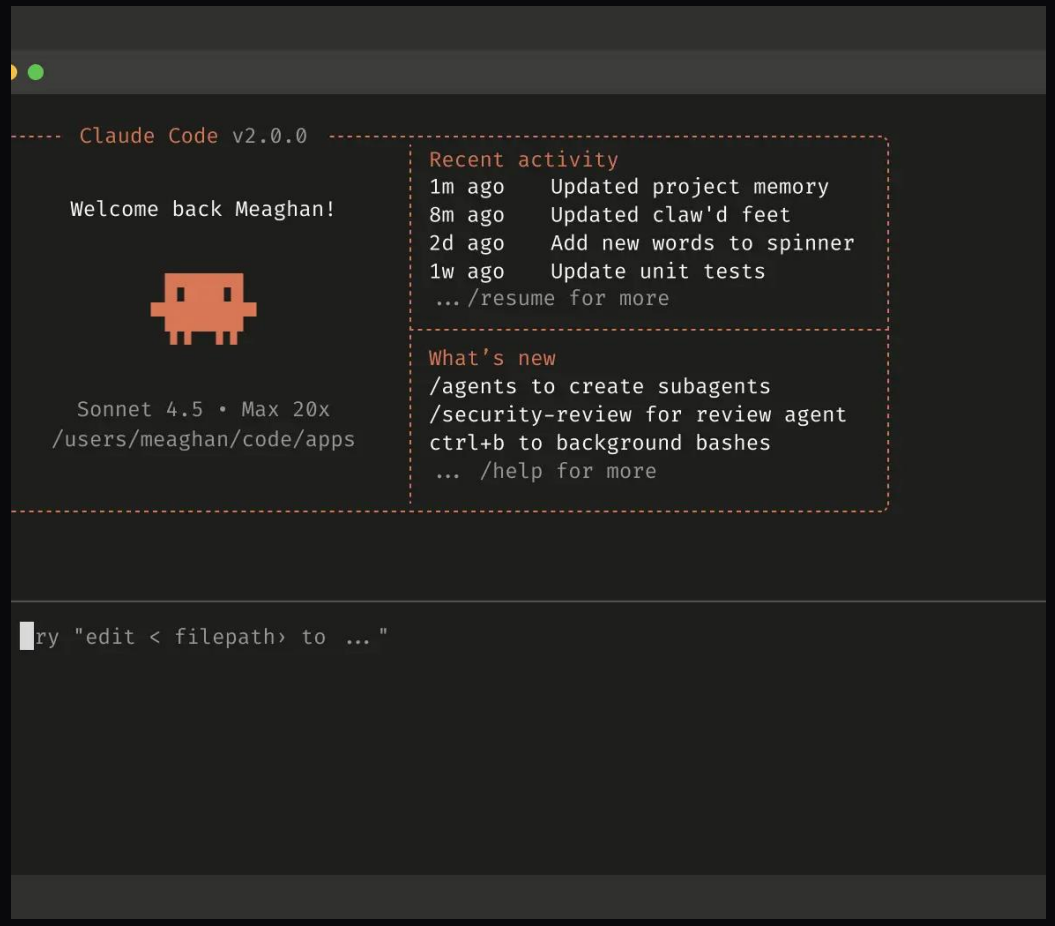
Back to Basics

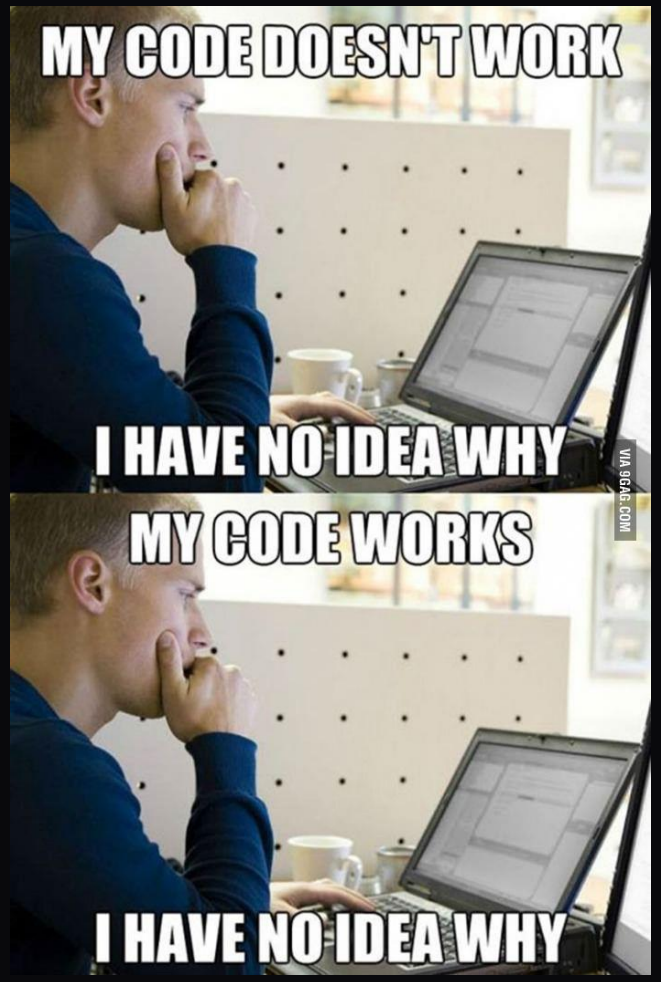
Creating Shopware Plugins from scratch

Developing in 2026 changed little bit





So we got from knowing hopefully the code to ...





Time machine: let's rewind to 1969



Wrong year, wrong universe. But who are you?

NAME	***Doraemon***
BORN	**1969**
ORIGIN	**Japanese Anime**
UNIVERSE	**Not ours (we overshot)**

And hey, they still build Shopware plugins here!



Shizuka wants a Product Badge Plugin for her store



Shizuka Minamoto

Owner of an online store

“Shopware already shows New and Sale. Can you build me a plugin for my own badges, like Bestseller or Limited? I want to set them per product in the admin and see them in my storefront.”

- Badge text per product
- Color per badge
- Shown in listing and detail page

BESTSELLER



Strawberry Shortcake

¥480

LIMITED



Sweet Potato Pie

¥320

What she wants to see in her storefront

A Shopware plugin is a Symfony bundle, plus extras

Your plugin

ProductBadge extends Plugin

Shopware\Core\Framework\Plugin

install, update, activate lifecycle hooks, migrations, config

Symfony Bundle

services.xml, DI container, event subscribers, routes

Each layer builds on the one below it

What lives inside a plugin

`composer.json`	type: shopware-platform-plugin, points to the class
`src/ProductBadge.php`	the plugin class, extends Plugin
`src/Resources/config/`	services.yaml, config.xml, routes
`src/Migration/`	database changes, versioned
`src/Resources/app/`	Administration and Storefront code (JS, Twig, SCSS)

composer.json makes the package a plugin and names it

```
{
  "name": "shopware/product-badge",
  "description": "Badges like New or Sale on products",
  "version": "1.0.0",
  "type": "shopware-platform-plugin",
  "license": "MIT",
  "autoload": { "psr-4": { "ProductBadge\\": "src/" } },
  "require": { "shopware/core": "~6.6.0" },
  "extra": {
    "shopware-plugin-class": "ProductBadge\\ProductBadge",
    "label": {
      "en-GB": "Product Badge",
      "de-DE": "Produkt-Badge"
    },
    "description": { "en-GB": "Show badges on products" }
  }
}
```

`type`

"shopware-platform-plugin" is what makes Composer and Shopware treat this package as a plugin. Without it: just a library.

`extra.shopware-plugin-class`

The fully qualified class Shopware instantiates. It must extend Plugin.

`extra.label`

Name shown in the admin plugin list, per locale. en-GB is the fallback; add de-DE for German admins.

The plugin class stays empty; lifecycle hooks are opt-in

```php
// src/ProductBadge.php
<?php declare(strict_types=1);

namespace ProductBadge;

use Shopware\Core\Framework\Plugin;

class ProductBadge extends Plugin
{
    // nothing here, and that is the point
}
```

Services, routes and templates are registered in config files, not here

Override only what you need; each runs once

`install()`	Seed defaults, e.g. badge types
`activate()` / `deactivate()`	Toggle what must not run while inactive
`update()`	Migrate data; schema goes in Migration/
`uninstall()`	Drop tables, unless keepUserData is set

Your services are not in the container yet. In install() and update() the plugin may be inactive and its bundle not booted, so `$this->container->get(MyService::class)` fails. Use what Shopware provides, e.g. `Connection` for SQL; your own services only from activate() on.

Where a plugin lives depends on where it comes from

`custom/plugins/`

Store extensions, manually

- Zipped plugins uploaded or copied in by hand
- Not managed by Composer: no lockfile, no dependency resolution
- Avoid it: install store extensions through Composer instead

Legacy path, still works

`custom/static-plugins/`

Project extensions

- Plugins written for this project, like ProductBadge
- Committed to the project repository
- Required via Composer path repository, so they still land in vendor/

Where our plugin goes

`vendor/`

Store extensions via Composer

- Installed from the Shopware Composer registry (store.shopware.com)
- One command: composer require store.shopware.com/<plugin>
- Versioned in composer.lock, updated like any other package

The recommended way

Where does the badge live: custom field or its own entity?



Doraemon
our developer

Shizuka picks a badge per product in the admin... but where do I save that? Is it just a field on the product, or a thing of its own?

Custom field

`CustomFieldSet on product`

- One JSON column already on the product
- Admin UI for free: a select on the product page
- No migration, no DAL definition, no entity class
- Hard to filter and join on; no validation of values

Right when the badge is just a label on a product

Own entity

`product_badge table, DAL definition`

- Own table and columns, validated by the DAL
- Filterable, sortable, joinable in criteria
- Reusable: one badge, many products; extra fields later
- Costs a migration, definition, collection and admin module

Right when the badge is a thing of its own

Custom fields are declared in one XML file, no code needed

```xml
// src/Resources/config/custom-fields.xml
<?xml version="1.0" encoding="utf-8"?>
<custom-fields xsi:noNamespaceSchemaLocation="…/custom-fields-1.0.xsd">
  <custom-field-set>
    <name>product_badge_set</name>
    <label>Product badge</label>
    <label lang="de-DE">Produkt-Badge</label>
    <related-entities><product/></related-entities>
    <fields>
      <select name="product_badge">
        <label>Badge</label>
        <position>1</position>
        <options><option value="new">New</option> …</options>
      </select>
    </fields>
  </custom-field-set>
</custom-fields>
```

Pink: the parts that matter for us. XML custom fields work since Shopware 6.7.13.0

Picked up on install

Shopware reads the file in install() and update() and creates the set. No PHP, no repository calls.

Set name is the key

product_badge_set must be unique across all plugins, so prefix it with your plugin name.

related-entities is where it shows

<product/> adds a Product badge tab to the product admin page; values land in product.customFields.

Field type picks the admin control

select gives a dropdown with fixed options; int, text, bool, date and more exist.

Doraemon goes looking for the Twig block that renders badges



Doraemon
our developer

The card already shows a New badge. Which template and block render it? Two ways to find out:

A **Search by CSS class or HTML**

Inspect the badge, grab a class like badge-new, grep the Storefront views for it.

B **Use FroshDevelopmentHelper**

Dev plugin wraps templates and blocks in HTML comments: the block name is in the page source.

vendor/shopware/storefront/…/component/product/card/badges.html.twig

```
{% block component_product_badges %}
    {% block component_product_badges_wrapper %}
        {% block component_product_badges_discount %}
            … Sale / % badge …
        {% endblock %}

        {% block component_product_badges_new %}
            {% if product.isNew %}
                <span class="badge bg-primary badge-new">
                    {{ "listing.boxLabelNew"|trans|sw_sanitize }}
                </span>
            {% endif %}
        {% endblock %}
    {% endblock %}
{% endblock %}
```

Every `{% block %}` is an extension point. Doraemon picks `component_product_badges_new`: his badge should sit right after the stock New badge.

The storefront badge extends one Twig block, not the template

```
// src/Resources/views/storefront/component/product/card/badges.html.twig
{% sw_extends '@Storefront/storefront/component/product/card/badges.html.twig' %}

{% block component_product_badges_new %}
    {{ parent() }}

    {% block component_product_badges_swag_product_badge %}
        {% set badge = product.translated.customFields.product_badge ?? null %}

        {% if badge %}
        <span class="badge bg-primary badge-swag-product-badge
              badge-swag-product-badge-{{ badge|e('html_attr') }}">
            {{ ('swagProductBadge.' ~ badge)|trans|sw_sanitize }}
        </span>
        {% endif %}
    {% endblock %}
{% endblock %}
```

Same path, sw_extends

Mirror the Storefront path under Resources/views and start with sw_extends. Shopware chains every plugin's version of the file; plain extends would break the chain.

Override a block, keep parent()

We hook into component_product_badges_new and call parent() first, so the stock New badge stays and ours is added after it.

Read the custom field

The select value from the XML sits in product.translated.customFields; the label comes from a snippet, swagProductBadge.<value>.

Doraemon delivers: Shizuka’s badges are live



Doraemon

Plugin developer, visiting from another universe

“Done! Pick Bestseller or Limited on any product in the admin, and the storefront shows it in the listing and on the detail page. One custom field, one Twig block.”

✓ Badge per product

✓ Color per badge

✓ Listing and detail page

BESTSELLER



Strawberry Shortcake

¥480

LIMITED



Sweet Potato Pie

¥320

Shizuka’s storefront today

Shizuka's next wish outgrows the custom field



Shizuka Minamoto

Owner of an online store

"Love it! Can I create my own badges, like Vegan or Gift idea? And put two or three on one product? Oh, and can my customers filter the listing by badge?"



"Uh... my one little select field can't do any of that. Now what?"

Own badges Vegan, Gift idea, anything

Custom field: Options are fixed in XML; a new badge needs a plugin update

Several per product Two or three at once

Custom field: A select stores one value, not a list

Filter the listing Customers narrow by badge

Custom field: JSON column: no index, no aggregation, no filter

Doraemon knows: the badge becomes its own entity



“Got it! The badge stops being a field on the product and becomes a thing of its own, with a table, an admin page and a storefront filter.”

The plan

- **Own entity:** product_badge with name, color, position
- **Link to products:** many-to-many: one product, many badges
- **Admin module:** list, create and edit badges; assign on the product
- **Storefront:** render all badges, filter listing by badge

1. **Entity definition + migration**
   ProductBadgeDefinition, Collection, Entity; CREATE TABLE product_badge
2. **Mapping to product**
   ProductBadgeMappingDefinition; extension on ProductDefinition
3. **Admin module**
   Vue module with list and detail pages; badge select on the product detail
4. **Storefront**
   Twig renders product.extensions.badges; listing filter via a criteria subscriber

TO BE CONTINUED...

...live, in the editor. Doraemon starts coding.

Next up: entity definition + migration • mapping • admin module • storefront